**PLANO DE TRABALHO**
**EA019**

| NOMES DO(S) ALUNO(S) PARTICIPANTE(S): | RA | CURSO |
|---|---|---|
| *1.Anderson Agondi dos Santos Paschoalon* | ***083233*** | *34AB* |
| *2.* | | |
| *3.* | | |

| | |
|---|---|
| DISCIPLINA: EA018 | |
| ORIENTADOR: Diedre Stefanelli do Carmo | |
| COORIENTADOR *(se houver):* | |
| TEMA DO PROJETO: Otimização do Uso de Recursos Computacionais Heterogêneos para Aprendizado Profundo | |
| RESUMO DO CRONOGRAMA DAS ATIVIDADES. | |
| PERÍODO DE REALIZAÇÃO: | DESCRIÇÃO**: |

| | | |
|---|---|---|
| **01-Novembro-2026** | **30-Novembro-2026** | **(1) Fechamento do escopo da metodologia e da análise experimental.**<br>***(2) Submissão do plano de trabalho*** |
| ***01-Dezembro-2026*** | ***15-Dezembro-2026*** | ***(1) Início de implementação da infraestrutura tecnológica proposta no plano.*** |
| ***01-Março-2027*** | ***01-Março-2027*** | ***(1) Implementação da infraestrutura tecnológica com escopo mais abrangente, envolvendo GPUs de diferentes granularidades de desempenho.***<br>***(2) Início da escrita do relatório final.*** |
| ***01-Abril-2027*** | ***30-Abril-2027*** | ***(1) Automatização completa da implementação da infraestrutura como código para submissão de jobs de aprendizado profundo.***<br>***(2) Início da realização de experimentos práticos quantificando experimentalmente os benefícios da metodologia proposta.***<br>***(3) Continuação da escrita do relatório final.*** |
| ***01-Maio-2027*** | ***30-Maio-2027*** | ***(1) Finalização da coleta dos resultados experimentais.***<br>***(2) Análise dos resultados.***<br>***(3) Finalização da escrita do relatório final.*** |
| ***01-Junho-2027*** | ***30-Junho-2027*** | ***(1) Finalização da análise experimental.***<br>***(2) Revisão geral.***<br>***(3) Entrega do relatório final.*** |

| | | |
|---|---|---|
| | | |

| |
|---|
| *DATA:* |
| *OBSERVAÇÕES:* |

** Para trabalhos em grupo cada participante deverá apresentar separadamente o formulário preenchido.*

INTRODUÇÃO:

A pesquisa em Aprendizado Profundo (Deep Learning) requer uso intensivo de recursos computacionais caros e escassos (sobretudo GPUs), por longos períodos de tempo. Isso naturalmente gera um atrito entre alunos e pesquisadores, pois, a partir do momento em que esses recursos não estão sobrando, se faz necessária a adoção de políticas de compartilhamento e agendamento de tarefas para que todos possam utilizá-los. Isso acaba por introduzir burocracia de planilhas e de controle manual de quem vai usar cada recurso e quando. Disto surgem dois problemas claros: primeiramente, como garantir que esse uso de recursos é otimizado e justo? Um tipo de organização manual pode gerar tanto uma tendência de adotar, de forma implícita, uma política de "primeiro que chegar, usa todo o recurso", ou então, em outro extremo, gerar uma burocracia que dificulte o acesso ao recurso. Em segundo lugar, mesmo que suponhamos que ele seja otimizado e justo, está completamente suscetível a falhas humanas. Imaginemos, por exemplo, que um pesquisador, por conta de algum contratempo, não possa submeter o seu trabalho no horário programado. Nessa situação o recurso ficaria ocioso, e o próprio pesquisador provavelmente iria para o "final da fila", atrasando sua entrega.

Visando endereçar essa situação, este projeto propõe implementar, no MICLab/LCA, um piloto de gerenciamento de recursos computacionais heterogêneos baseado na adoção de uma pilha tecnológica open source em torno de um escalonador de jobs e ferramentas auxiliares. Após a avaliação e um estudo de diversas ferramentas disponíveis para esse trabalho (como, por exemplo, o Slurm e o SGE), optamos pelo Slurm como escalonador, por ser uma ferramenta madura e com adoção amplamente consolidada em ambientes HPC (High-Performance Computing). Além disso, toda a infraestrutura será provisionada de forma automatizada utilizando o conceito de "Infrastructure as Code" (IaC). Como coluna vertebral do IaC, optou-se pelo uso do Ansible, por sua flexibilidade em comparação com outras tecnologias e por conformidade com a infraestrutura do laboratório. Além do Ansible, podemos também citar YAML para configuração, Jinja para templates,

e Python e Shell para a implementação de scripts e automações. Nessa infraestrutura, o armazenamento compartilhado será centralizado em um NAS (Network-Attached Storage), cuja tecnologia específica (NFS ou Ceph) ainda está em definição.

Como caso de estudo prático, visando a validação da infraestrutura, bem como estudos de distribuição de carga e fair-share, serão utilizados treinamentos de aprendizado profundo de visão computacional com diferentes granularidades de carga, como fine-tuning de diferentes modelos — como, por exemplo, o RepVGG —, que servirão como padrão de benchmark.

Por fim, este trabalho se apoia no problema de escalonamento descrito por Cormen et al. (Problema 16-2, *Introduction to Algorithms* — "Scheduling to minimize average completion time") como fundamentação teórica do que se entende por escalonamento eficiente. A partir dessa fundamentação, pretende-se demonstrar empiricamente, em que medida o escalonamento do Slurm se aproxima desse regime eficiente teórico, ao mesmo tempo que implementamos uma distribuição "justa" (fair-share).

**Comparação de escalonadores open-source avaliados**

A pilha final foi escolhida comparando os escalonadores open-source disponíveis segundo os quatro critérios que realmente importam para este trabalho: (1) justiça (fair-share), (2) uso eficiente de recursos, (4) facilidade de automatizar via IaC, e (4) ser open-source.

- ☐ **Slurm:**
  ***(1) fair-share nativo;***
  ***(2) uso de GPU maduro e eficiente;***
  ***(3) fácil de automatizar via IaC (módulos Ansible consolidados);***
  ***(4) open-source ativo (SchedMD/NVIDIA).***
- ☐ **OpenPBS:**
  ***(1) fair-share via filas configuráveis;***
  ***(2) bom uso de recursos;***
  (3) IaC possível, porém menos padronizado;
  (4) open-source, com versão paga à parte.
- ☐ **HTCondor:**
  (1) sem fair-share real (negociação por job);
  (2) pensado para jobs isolados, não para GPU compartilhada;
  ***(4) open-source.***
- ☐ **Grid Engine (SGE):**
  (1) já teve fair-share forte;
  (2) suporte a GPU datado;
  (4) ecossistema open-source fragmentado desde 2010.

Dos quatro critérios acima, o Slurm é o único que atende bem a todos ao mesmo tempo, por isso foi o escolhido.

OBJETIVOS:

O projeto visa implantar um piloto de gerenciamento de recursos computacionais heterogêneos, incluindo GPUs comumente usadas em experimentos de aprendizado profundo, utilizando uma pilha tecnológica open-source (Slurm) provisionada via IaC (Ansible), de modo a melhorar o aproveitamento da infraestrutura disponível no laboratório MICLab/LCA por pesquisadores de graduação e pós-graduação. Os cinco objetivos específicos são:

- **Reduzir subutilização:** otimizar o uso de recursos computacionais existentes, em especial GPUs e computadores mais modestos;
- **Viabilizar computação distribuída:** possibilitar experimentos com recursos distribuídos, buscando novos níveis de desempenho a partir do hardware já existente;
- **Acelerar a execução de experimentos:** reduzir o tempo de espera e de execução dos experimentos dos usuários (alunos) por meio de uma melhor gestão de compartilhamento de recursos;
- **Aumentar a eficiência computacional:** melhorar a disponibilidade de capacidade de processamento sem necessidade de investimento adicional em hardware;
- **Garantir equidade no acesso:** assegurar que o acesso à capacidade de processamento ocorra de forma justa entre os usuários, por meio da implementação de uma fila de execução de experimentos (fair-share).

A validação de que esses objetivos foram de fato alcançados será feita por meio dos experimentos comparativos descritos em Resultados Esperados, com especial atenção à comparação entre o escalonamento do Slurm e uma fila ingênua (FCFS), que representa o limite teórico superior de um laboratório sem escalonador algum.

CRONOGRAMA DE ATIVIDADES DETALHADO:

Setembro/2026

- Estudo da bibliografia levantada (Slurm, SGE/Grid Engine, HTCondor, OpenPBS, IaC, fair-share) e início das atividades exploratórias
- Levantamento de treinamentos de deep learning que podem ser usados como benchmark (ex.: candidatos a fine-tuning, como RepVGG)

Outubro/2026

- Fechamento do escopo da pilha tecnológica: comparação formal entre Slurm, OpenPBS, HTCondor e Grid Engine (ver seção de Comparação de escalonadores acima) e decisão final pelo Slurm
- Definição dos experimentos de infraestrutura a serem realizados
- Escrita do Plano de Trabalho

Novembro/2026

- Fechamento do escopo da metodologia e da análise experimental
- Submissão do Plano de Trabalho

Dezembro/2026

- Início da implementação da infraestrutura tecnológica proposta no plano (primeiro provisionamento via Ansible: Slurm controller + MariaDB)

Março/2027

- Implementação da infraestrutura com escopo mais abrangente, envolvendo GPUs de diferentes granularidades de desempenho; definição final do protocolo de armazenamento compartilhado (NFS ou Ceph) para o NAS já existente no laboratório
- Início da escrita do relatório final

Abril/2027

- Automatização completa da implementação da infraestrutura como código (IaC) para submissão de jobs de aprendizado profundo
- Início da realização dos experimentos práticos: execução do fine-tuning real (prova de conceito) e coleta de dados comparando os regimes de escalonamento definidos na Metodologia Experimental
- Continuação da escrita do relatório final

Maio/2027

- Finalização da coleta dos resultados experimentais
- Análise dos resultados (geração dos gráficos comparativos)
- Finalização da escrita do relatório final

Junho/2027

- Finalização da análise experimental
- Revisão geral
- Entrega do relatório final

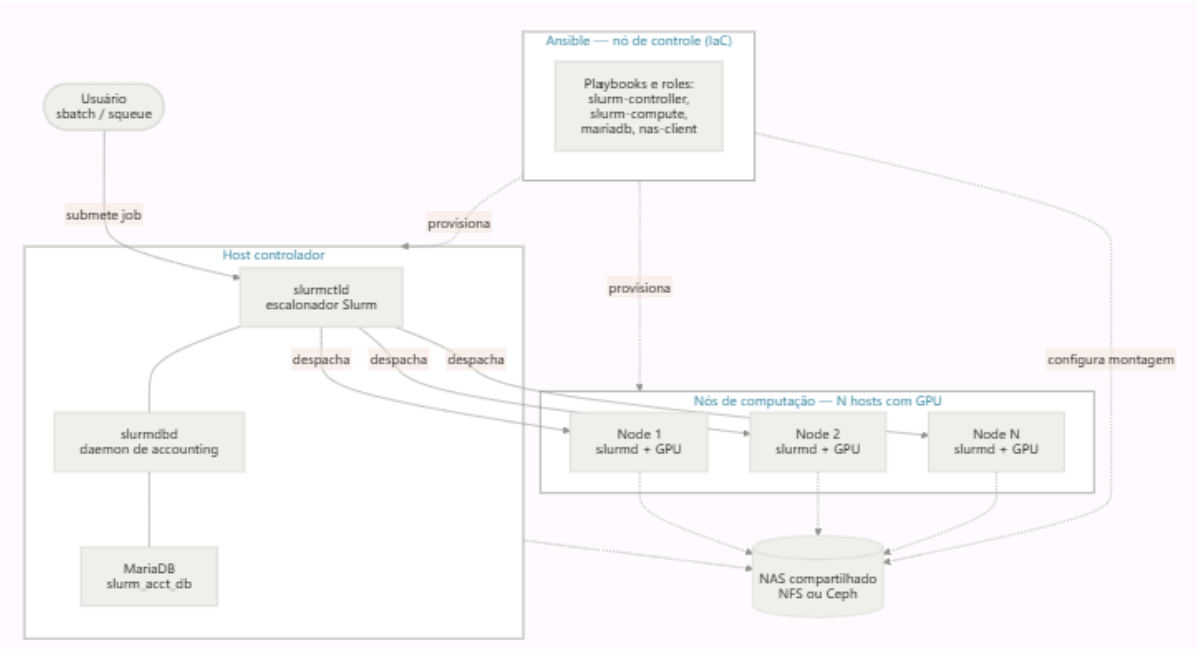
RESULTADOS ESPERADOS:

Podemos dividir os resultados esperados do projeto em quatro pilares principais, cada um abordando diferentes aspectos teóricos e práticos da solução:

- **Fundamentação teórica e revisão bibliográfica:** Fundamentar as escolhas do projeto por meio de uma análise rigorosa de requisitos e da revisão da literatura. Nossa análise prévia para este documento já foi apresentada na Introdução e na Comparação de Escalonadores, não sendo necessário reapresentá-la nesta seção.
- **Automação completa no provisionamento da infraestrutura**: Demonstrar que nossa implementação baseada em Infrastructure as Code (IaC) é capaz de provisionar a implantação da infraestrutura com mínima intervenção manual.
- **Prova de conceito**: Execução bem-sucedida de pipelines reais de treinamento de Aprendizado Profundo (Deep Learning) sobre a infraestrutura provisionada.
- **Comprovação empírica da eficácia das políticas de escalonamento:** Comprovar, por meio da execução de treinamentos reais, que o escalonamento do Slurm atinge uma taxa de utilização que se aproxima do teto teórico de eficiência (Shortest Processing Time First: SPT). Confirmar também que o Slurm, por sua política de fair-share, realiza uma distribuição de recursos mensuravelmente mais justa do que uma fila ingênua First-Come, First-Served (FCFS)

**Automação completa no provisionamento da infraestrutura**

Na figura a seguir [inserir referência da figura], apresentamos o diagrama proposto para a topologia da infraestrutura do projeto.




A infraestrutura necessária para a execução dos experimentos científicos é composta por um nó controlador (controller host) que abriga os serviços essenciais do Slurm:

- **Aplicações cliente**: Utilitários de linha de comando (sbatch, squeue, entre outros);
- **Escalonador central (slurmctld)**: Encarregado de gerenciar as filas e provisionar a alocação dos jobs;
- **Serviço de contabilidade (slurmdbd)**: Responsável por registrar métricas de uso, dados de execução e configurações de prioridade;
- **Base de dados relacional (MariaDB)**: Utilizada para o armazenamento persistente do histórico mantido pelo slurmdbd;
- **Nós de computação (workers)**: $N$ nós equipados com aceleradores gráficos (GPUs) executando o daemon slurmd;
- **Armazenamento compartilhado (NAS)**: Centralizador de dados e arquivos de experimentos acessível por todos os nós

Provisionar manualmente toda essa infraestrutura heterogênea é uma tarefa altamente complexa e sujeita a erros, consumindo um tempo expressivo. É justamente nesse cenário que se aplicam os princípios de Infrastructure as Code (IaC):

- Centralização da configuração: Única fonte da verdade, eliminando a redundância de informações;
- Instanciação automatizada de recursos: Automação do ciclo de vida dos serviços e dependências;
- Reprodutibilidade e escalabilidade: Replicação simples e idêntica em diferentes ambientes de laboratório.

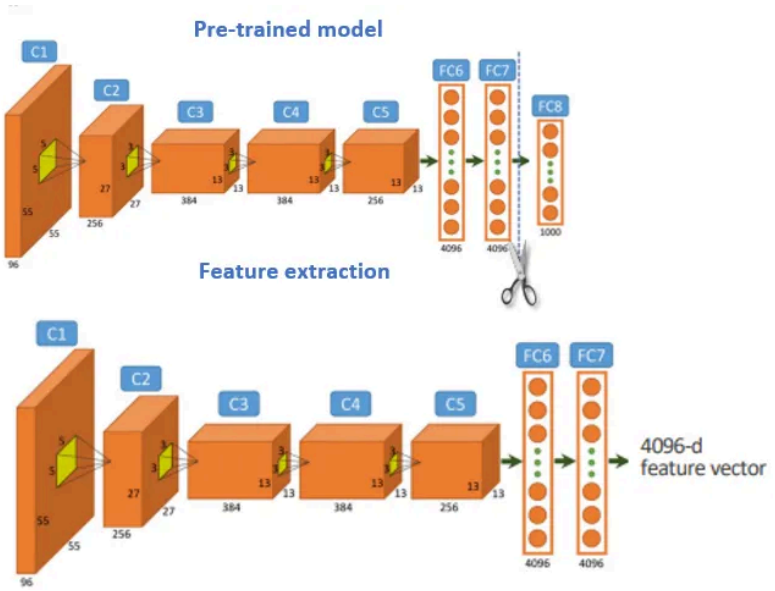
Os principais entregáveis e resultados esperados desta etapa serão:

- Criação de um repositório no GitHub contendo toda a solução de automação do processo de construção e configuração da infraestrutura;
- Elaboração de um tutorial claro e objetivo sobre como provisionar um ambiente de cluster Slurm a partir do zero;
- Demonstração prática de que a infraestrutura completa de um novo laboratório pode ser instanciada de forma automatizada em poucos minutos.

**Prova de Conceito: Treinamentos com Aprendizado Profundo**

Para validar a estabilidade e a funcionalidade da infraestrutura computacional provisionada, o projeto empregará a execução de treinamentos reais de aprendizado profundo no domínio de Visão Computacional, que possui especial relevância para o MICLab/LCA. Como caso de estudo prático, adotaremos a técnica de fine-tuning de arquiteturas consolidadas de redes neurais convolucionais, tomando como modelo de referência a arquitetura RepVGG.

O processo de fine-tuning consiste em reaproveitar os pesos pré-treinados de uma rede de grande porte, e adaptá-la para um novo conjunto de imagens com contexto específico de interesse do laboratório. Em vez de inicializar os parâmetros do modelo de forma aleatória, o que exigiria tempo proibitivo de processamento e um volume massivo de dados, congelam-se ou ajustam-se as camadas iniciais (encarregadas de extrair características genéricas, como bordas e texturas) e retreinam-se as camadas superiores com uma taxa de aprendizado (learning rate) reduzida. Para maximizar a capacidade de generalização e evitar o sobreajuste (overfitting) diante de conjuntos de dados específicos ou limitados, aplicam-se técnicas de aumento de dados (data augmentation), tais como rotações aleatórias, recortes, variações de contraste e iluminação.




No contexto deste projeto de TCC, a execução do fine-tuning do RepVGG desempenha um papel duplo: além de representar uma carga de trabalho científica relevante para o laboratório, serve também como métrica de benchmark de infraestrutura. A execução desses treinamentos no cluster gerido pelo Slurm permitirá demonstrar e validar empiricamente:

- A correta alocação e o isolamento de hardware (GPUs e memória) por job via cgroups;
- A estabilidade do sistema de arquivos compartilhado (NAS) durante a leitura continuada de lotes (batches) de imagens pelos nós de computação;
- O registro preciso de métricas de uso e tempo de execução na base de dados do Slurm (slurmdbd e MariaDB);
- A capacidade do escalonador em suportar execuções de longa duração sem interrupções ou degradação de desempenho.

**Comprovação empírica da eficácia das políticas de escalonamento**

**Embasamento Teórico**

Esta seção aborda referências teóricas para validação da metodologia experimental do projeto, e se apoia em duas teses:

1. O escalonamento do Slurm se aproxima de um regime eficiente teórico;

2. O Slurm é mais justo do que um escalonador naive.

As duas serão comprovadas executando treinamentos reais de aprendizado profundo no cluster Slurm implantado — os testes com dados simulados (mockados) apresentados a seguir são apenas um exercício preliminar, para antecipar o que devemos encontrar, e não uma prova.

O teto teórico: o modelo de Shortest Processing Time First (SPT)

O problema de Cormen et al. (Problema 16-2 "Scheduling to minimize average completion time") formaliza um conjunto S de n tarefas $a_1,...,a_n$, cada uma com um tempo de processamento $p_i$, rodando em um único processador. Sendo $c_i$ o tempo de conclusão da tarefa $a_i$, o objetivo é minimizar o tempo médio de conclusão:

minimizar $(1/n) \Sigma c_i \equiv$ minimizar $\Sigma c_i$

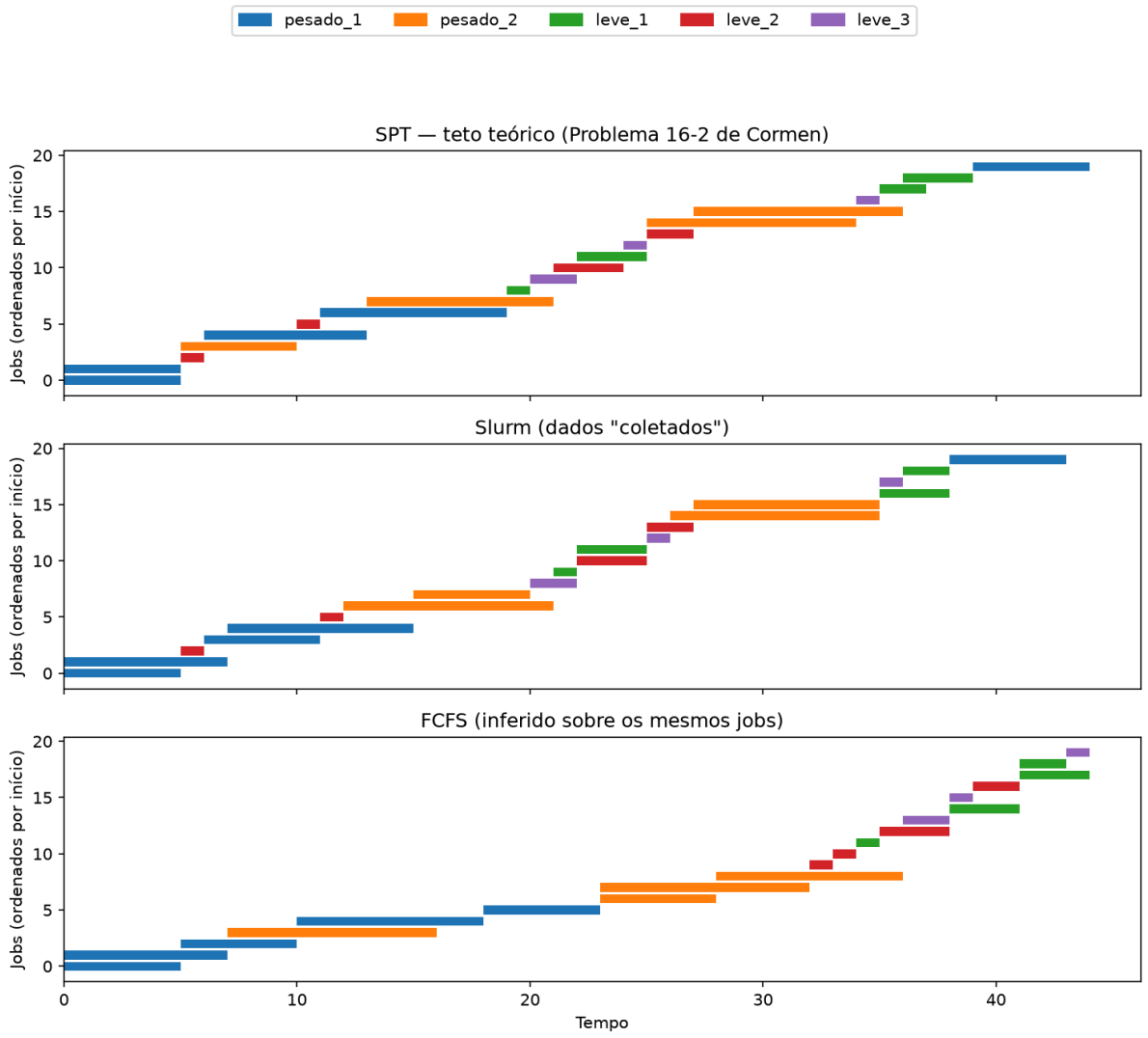
A resposta clássica é rodar as tarefas em ordem crescente de duração — Shortest Processing Time First (SPT). O tempo de conclusão de cada tarefa é sempre o tempo de chegada mais o tempo de espera mais a própria duração; como a duração e o instante de chegada de cada job não dependem de qual escalonador está sendo usado — são propriedades do job, não do escalonador —, a única coisa que muda entre os regimes testados é o tempo de espera. Por isso, minimizar a soma dos tempos de conclusão é equivalente a minimizar a soma dos tempos de espera. O SPT representa, portanto, o teto teórico: o menor tempo de espera total que um escalonador poderia, em tese, alcançar sobre o mesmo conjunto de jobs.

A justiça segundo o Slurm

O Slurm, por padrão, prioriza jobs através do Multifactor Priority Plugin, cuja fórmula (SCHEDMD, documentação oficial) é:

```
Prioridade = site_factor + (PriorityWeightAge × fator_idade) + (PriorityWeightFairshare × fator_fairshare) + (PriorityWeightJobSize × fator_tamanho) + (PriorityWeightPartition × fator_partição) + (PriorityWeightQOS × fator_QOS) + Σ(PriorityWeightTRES × fator_TRES) − fator_nice
```

O termo que nos interessa é o fator de fair share: ele dá mais prioridade a quem consumiu menos recursos recentemente, com o uso decaindo no tempo (SCHEDMD, Fair Tree Fairshare Plugin). É essa noção — tratar melhor quem historicamente usou menos — que define o que chamamos de "justo" neste trabalho, e não qualquer definição de equidade proveniente de Cormen.




**Simulação Teórica Preliminar**

A seguir, foi implementado um exercício experimental com valores simulados, a fim de testarmos preliminarmente as teses que queremos validar.

• Foram criados 5 diferentes usuários simulados. Os dois primeiros "pesados" possuíam jobs pesados para serem realizados;

• Os outros 3, "leves", possuíam somente jobs leves para serem realizados;

• A ordem de submissão de cada um desses jobs foi sorteada;

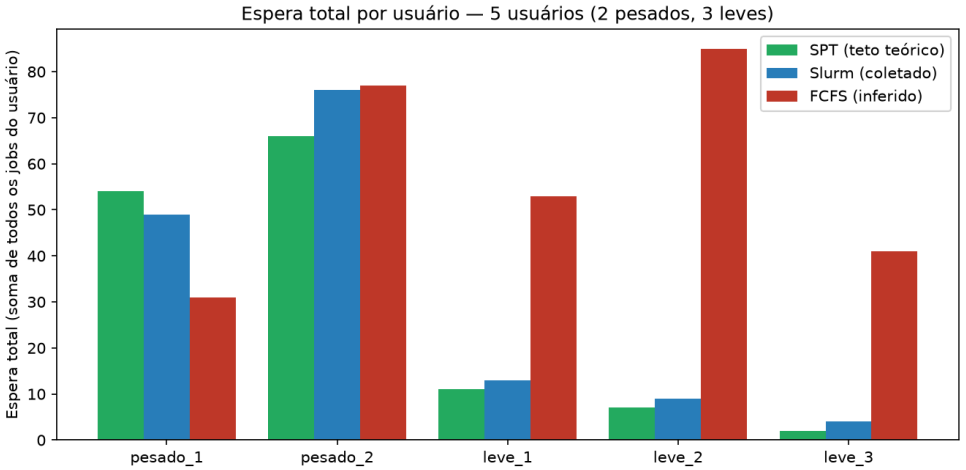
• Tendo em vista a ordem de submissão e o tempo de execução de cada job, fizemos 3 simulações diferentes:

◦ Uma fila ingênua FCFS (First-Come, First-Served), que representa o limite teórico superior de um laboratório sem escalonador; ou seja, não é possivel um laboratório sem escalonador ter um resultado superior a este.

◦ A ordem gerada pela simulação do Slurm (fair-share + backfill);

◦ O limite teórico inferior, dado pelo SPT (Shortest Processing Time First — "menor duração primeiro"), no qual as tarefas são executadas na ordem ótima de eficiência, já conhecendo a priori a duração de cada uma. Corresponde ao limite de menor duração dos tempos de espera acumulados, sem se preocupar com distribuição justa de cargas.

Seguem abaixo os resultados da simulação. Mesmo que elas ainda não represente o que de fato iremos ver em execução na realidade, elas servem para já termos uma ideia do tipo de resultado que esperamos, bem como serve também para validar a visualização dos resultados proposta.




Nesse exercício com 5 usuários, a soma total dos tempos de espera foi de 140 unidades no SPT (o limite teórico minimo), 151 no Slurm simulado, e 287 no FCFS. Uma vez que o método SPT se preocupa apenas com a minimização do tempo esperado, e não com uma divisão justa das janelas de tempo, o resultado simulado saiu exatamente como esperado. O Slurm se aproximou do limite teórico de máxima eficiência (mas não alcançou), e ficou melhor que o algorítimo ingênuo de escalonamento (FCFS). exatamente a ordem esperada: o Slurm ficou entre o pior caso (sem escalonador) e o melhor caso possível.

Quanto à justiça, comparamos a espera média por job dos dois grupos: sob o FCFS, os pesados esperavam em média 12,00 unidades de tempo por job e os leves 16,27; sob o Slurm simulado, os pesados passaram a esperar 13,89 e os leves apenas 2,36 — uma redução de 85,5% para os leves, com os pesados praticamente estáveis. É um indício de que o Slurm protege quem historicamente é sub-atendido, mas, de novo, isso ainda precisa ser confirmado com os treinamentos reais.

| Caso | Espera total (soma) | Espera média — gulosos | Espera média — magros |
|---|---|---|---|
| SPT (limite teórico inferior) | 140 | 13,33 | 1,82 |
| Slurm (simulado) | 151 | 13,89 | 2,36 |
| FCFS (limite teórico sem escalonador) | 287 | 12,00 | 16,27 |

**Conclusão**

A execução deste plano de trabalho visa entregar não apenas uma solução técnica pontual, mas uma infraestrutura de computação de alto desempenho robusta, automatizada e cientificamente validada para o ambiente de laboratório.

O projeto consolidará a escolha da arquitetura Slurm mediante uma análise comparativa frente a outros gerenciadores de recursos open-source (como SGE, OpenPBS/PBS Pro e HTCondor), culminando na implantação de um cluster funcional.

A adoção de Infrastructure as Code (IaC) via Ansible garantirá a reprodutibilidade rápida e automatizada da infraestrutura, devidamente integrada ao banco de dados MariaDB para contabilidade de uso (accounting) e a um sistema de armazenamento compartilhado via NAS.

A validação prática do ambiente será comprovada por meio da execução de pipelines reais de fine-tuning do modelo de visão computacional RepVGG, assegurando que o cluster suporta de forma estável e isolada cargas de trabalho intensivas de Aprendizado Profundo.

Por fim, a avaliação experimental demonstrará empiricamente que o escalonamento do Slurm se aproxima do teto teórico de eficiência (modelo SPT de Cormen et al.) ao mesmo tempo em que promove uma distribuição de recursos mensuravelmente mais justa que uma fila FCFS, reduzindo de forma expressiva o tempo de espera de tarefas leves. Dessa forma, a integração entre automação de sistemas, validação prática e análise quantitativa de desempenho assegurará o cumprimento integral de todos os objetivos propostos para o TCC.

REFERÊNCIAS BIBLIOGRÁFICAS:


- CORMEN, T. H. et al. *Introduction to Algorithms*, 3ª ed. — Capítulo 16, Problema 16-2 (Scheduling to minimize average completion time)
- GÉRON, Aurélien. *Mãos à obra: aprendizado de máquina com Scikit-Learn, Keras & TensorFlow*. 2. ed. Rio de Janeiro: Alta Books, 2021
- SCHEDMD. *Slurm Workload Manager*: Documentation. [S. l.]: SchedMD, 2026. Disponível em: https://slurm.schedmd.com/documentation.html. Acesso em: 21 ago. 2026
- SCHEDMD. *Fair Tree Fairshare Plugin*: Slurm Workload Manager. [S. l.]: SchedMD, 2026. Disponível em: https://slurm.schedmd.com/fair_tree.html. Acesso em: 21 ago. 2026
- NVIDIA. *NVIDIA DeepOps*: Deployment guide for GPU clusters. [S. l.]: NVIDIA, 2024. Disponível em: https://github.com/NVIDIA/deepops. Acesso em: 21 ago. 2026
- MAJEE, Arindam. *DeepOps & SLURM*: Your GPU cluster guide. arXiv preprint arXiv:2405.00030, 2024
- WANG, Farui; ZHANG, Weizhe; LAI, Shichao; HAO, Meng; WANG, Zheng. Dynamic GPU energy optimization for machine learning training workloads. *IEEE Transactions on Parallel and Distributed Systems*, v. 33, n. 11, p. 2943–2954, 2022
- GENTZSCH, Wolfgang. Sun grid engine: towards creating a compute power grid. *Proceedings First IEEE/ACM International Symposium on Cluster Computing and the Grid*. IEEE, 2001
- [adicionar: documentação oficial do Ansible; documentação do MariaDB; referência do protocolo de NAS escolhido (NFS ou Ceph); paper do modelo de fine-tuning utilizado, se for RepVGG ou outro]
- SCHEDMD. Multifactor Priority Plugin: Slurm Workload Manager. Disponível em: https://slurm.schedmd.com/priority_multifactor.html
- HPCWIRE. NVIDIA Acquires Open-Source Workload Management Provider SchedMD. Dez. 2025. Disponível em: https://www.hpcwire.com/off-the-wire/nvidia-acquires-open-source-workload-management-provider-schedmd/